

Flexbox layout

In one of the previous lessons, you used `display: flex` just enough to make `gap` work. Now it is time to learn what Flexbox is actually doing.

Flexbox is a layout mode for arranging items in one direction at a time: across a row or down a column. It is useful when a group of items needs spacing, alignment, wrapping, or equal sharing of available space.

Think about the layouts you have already seen:

- A header with text on the left and an action on the right.
- A row of filter buttons.
- A group of cards that should sit beside each other when there is room.
- A small card footer where metadata and a link should line up.

Those are the kinds of problems Flexbox is designed to solve.

Start with the layout problem

A page can have good colors, spacing, borders, and states, but still feel awkward if grouped items do not line up.

In our example webpage below, the header action should sit beside the title, the filter buttons should have consistent space between them, and the lesson cards should wrap when the screen gets narrow.

Use this HTML as `index.html` :

html

```
<!DOCTYPE html>
<html lang="en">
  <head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1">
    <title>Flexbox practice</title>
    <link rel="stylesheet" href="style.css">
  </head>
  <body>
    <main class="page">
      <header class="page-header">
        <div>
          <p class="eyebrow">CSS practice</p>
          <h1>Plan your next CSS session</h1>
          <p class="summary">Flexbox helps groups of items line up, share space, and wrap</p>
        </div>

        <a class="header-action" href="/new">New lesson</a>
      </header>
```



```
<section class="toolbar" aria-label="Lesson filters">
  <button class="filter is-active" type="button">All</button>
  <button class="filter" type="button">Layout</button>
  <button class="filter" type="button">Practice</button>
  <button class="filter" type="button">Review</button>
</section>

<section class="lesson-list">
  <article class="lesson-card">
    <p class="card-label">Layout</p>
    <h2>Flexbox basics</h2>
    <p>Use a row or column layout when a group of items needs alignment and spacing.</p>
    <div class="card-footer">
      <span>7 min</span>
      <a class="card-link" href="/flexbox">Open</a>
    </div>
  </article>

  <article class="lesson-card">
    <p class="card-label">Practice</p>
    <h2>Spacing decisions</h2>
    <p>Compare padding, margin, and gap so each kind of space has a clear job.</p>
    <div class="card-footer">
      <span>9 min</span>
      <a class="card-link" href="/spacing">Open</a>
    </div>
  </article>

  <article class="lesson-card">
    <p class="card-label">Review</p>
    <h2>DevTools check</h2>
    <p>Inspect the final layout and confirm which elements became flex items.</p>
    <div class="card-footer">
      <span>5 min</span>
      <a class="card-link" href="/devtools">Open</a>
    </div>
  </article>
</section>
</main>
</body>
</html>
```

Add this starting CSS to `style.css` . It uses styling you already know: colors, spacing, borders, rounded corners, and button-like links. The layout is still

CSS

```
* {
  box-sizing: border-box;
}

body {
  font-family: system-ui, sans-serif;
  color: #111827;
  background-color: #f9fafb;
}

.page {
  max-width: 920px;
  margin: 40px auto;
}

.page-header,
.lesson-card {
  background-color: white;
  border: 1px solid #e5e7eb;
  border-radius: 8px;
  padding: 24px;
}

.eyebrow,
.card-label {
```

```
color: #2563eb;
font-size: 14px;
font-weight: 700;
}

h1,
h2,
p {
  margin-top: 0;
}

.summary {
  color: #4b5563;
  line-height: 1.6;
  max-width: 560px;
}

.header-action,
.filter,
.card-link {
  border-radius: 6px;
  font-family: inherit;
  font-size: 16px;
  font-weight: 700;
}

.header-action,
.card-link {
  color: #2563eb;
  text-decoration-thickness: 2px;
  text-underline-offset: 3px;
}

.header-action {
  display: inline-block;
  color: white;
  background-color: #2563eb;
  padding: 10px 14px;
  text-decoration: none;
}

.toolbar {
  margin: 24px 0;
}

.filter {
  color: #2563eb;
  background-color: white;
  border: 1px solid #bfdbfe;
  padding: 10px 14px;
  cursor: pointer;
}

.filter.is-active {
  color: white;
  background-color: #2563eb;
  border-color: #2563eb;
}

.lesson-list {
  margin-top: 24px;
}

.lesson-card {
  margin-bottom: 16px;
}

.card-footer {
  margin-top: 18px;
  color: #6b7280;
  font-size: 14px;
}
```

Open the page in your browser. The header action sits after the text, the buttons depend on normal inline spacing, and the cards stack in one column.

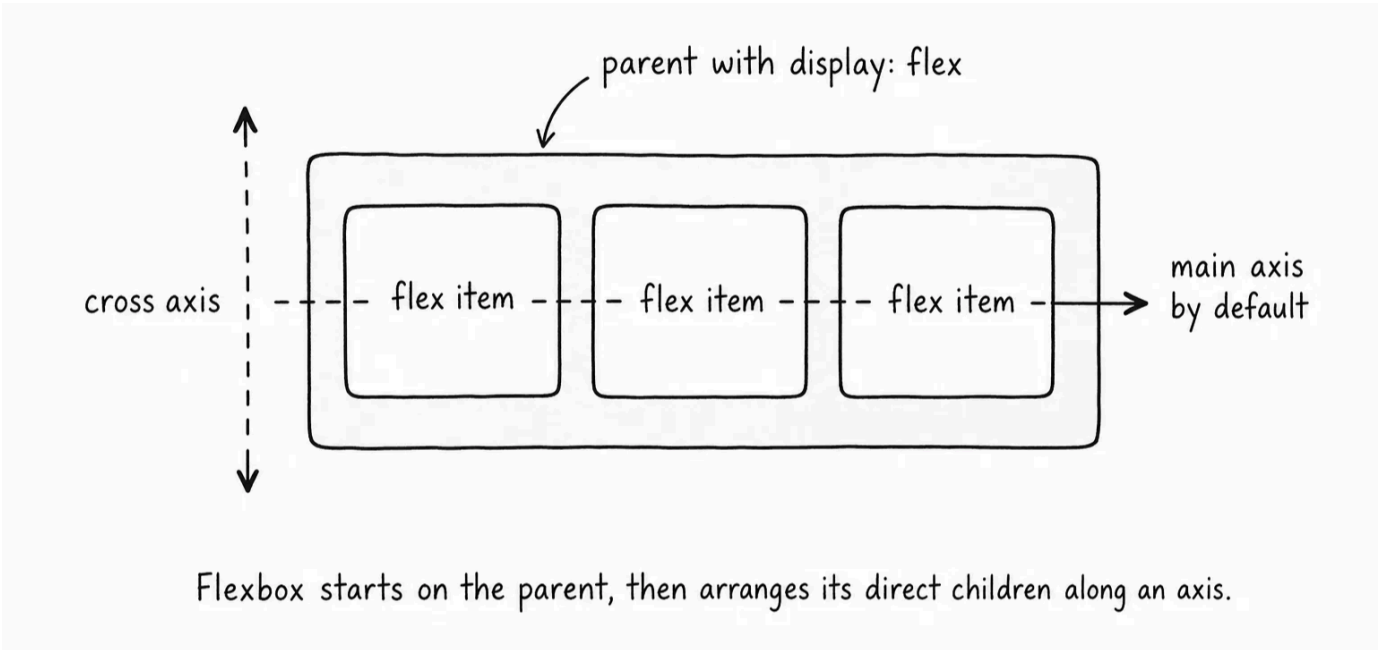
Nothing is broken, but the page is not using layout intentionally yet.

Flexbox starts on the parent

Flexbox is a parent-child layout mode. You put `display: flex` on a parent element, and the direct children of that parent become flex items.

That direct-child detail matters. In our webpage, if you put Flexbox on `.lesson-list`, the `.lesson-card` elements become flex items. If you put Flexbox on `.card-footer`, the time label and link inside that footer become flex items.

By default, when you apply `display: flex` on an element, flexbox arranges the direct children in a row. That row is called the **main axis**. The opposite direction is called the **cross axis**.



Put the header on one row

The header has two direct children: the text group and the `New lesson` link. Those two pieces belong on the same row, with space between them.

Add Flexbox to the header:

```
css

.page-header {
  display: flex;
  justify-content: space-between;
  align-items: flex-start;
  gap: 16px;
}
```

`display: flex` turns `.page-header` into a flex container. Its two direct children now sit in a row.

`justify-content` controls the main axis. In the default row direction, that means horizontal distribution. `space-between` pushes the first item to the start, the last item to the end, and leaves the extra space between them.

align-items controls the cross axis. In a row, that means vertical alignment.

flex-start keeps the action link aligned near the top of the header instead of stretching or floating awkwardly.

gap creates space between flex items. It is safer than adding margin to only one item because the spacing belongs to the group, not to a specific child.

 **Read Flexbox from the parent** - When you see a Flexbox rule, first ask: which parent has **display: flex**, and which direct children is it arranging?

Use gap for button rows

The filter buttons are a group. The space between them should belong to the group, so **gap** is a better fit than random margins on each button.

Add Flexbox to **.toolbar** :

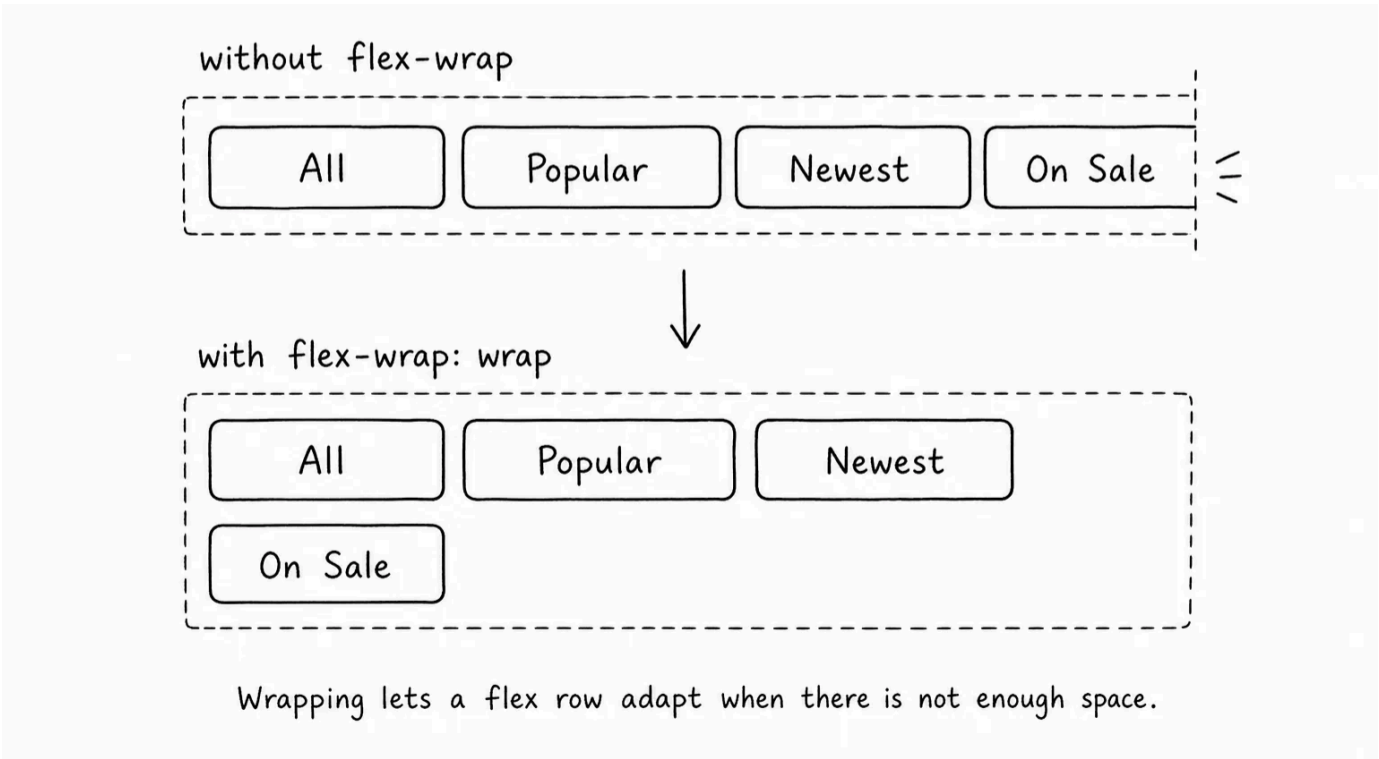
css

```
.toolbar {
  display: flex;
  gap: 8px;
  flex-wrap: wrap;
  margin: 24px 0;
}
```

display: flex makes the buttons flex items. **gap: 8px** creates consistent space between them.

flex-wrap: wrap lets items move onto a new line when there is not enough horizontal space. Without wrapping, Flexbox tries to keep items on one line, which can make a row feel cramped or overflow on smaller screens. In the current HTML, you may not feel the impact immediately, so try adding more items to **.toolbar** and compare the row with and without **flex-wrap** .

Alternatively, resize the browser window. The filter buttons should wrap more gracefully when the row runs out of space.



Let cards share available space

The lesson cards currently sit in a vertical list. They should sit beside each other when there is room, but they should not become too narrow. This is where Flexbox becomes more useful than fixed widths.

Add Flexbox to the card list:

```
CSS

.lesson-list {
  display: flex;
  gap: 16px;
  flex-wrap: wrap;
  margin-top: 24px;
}

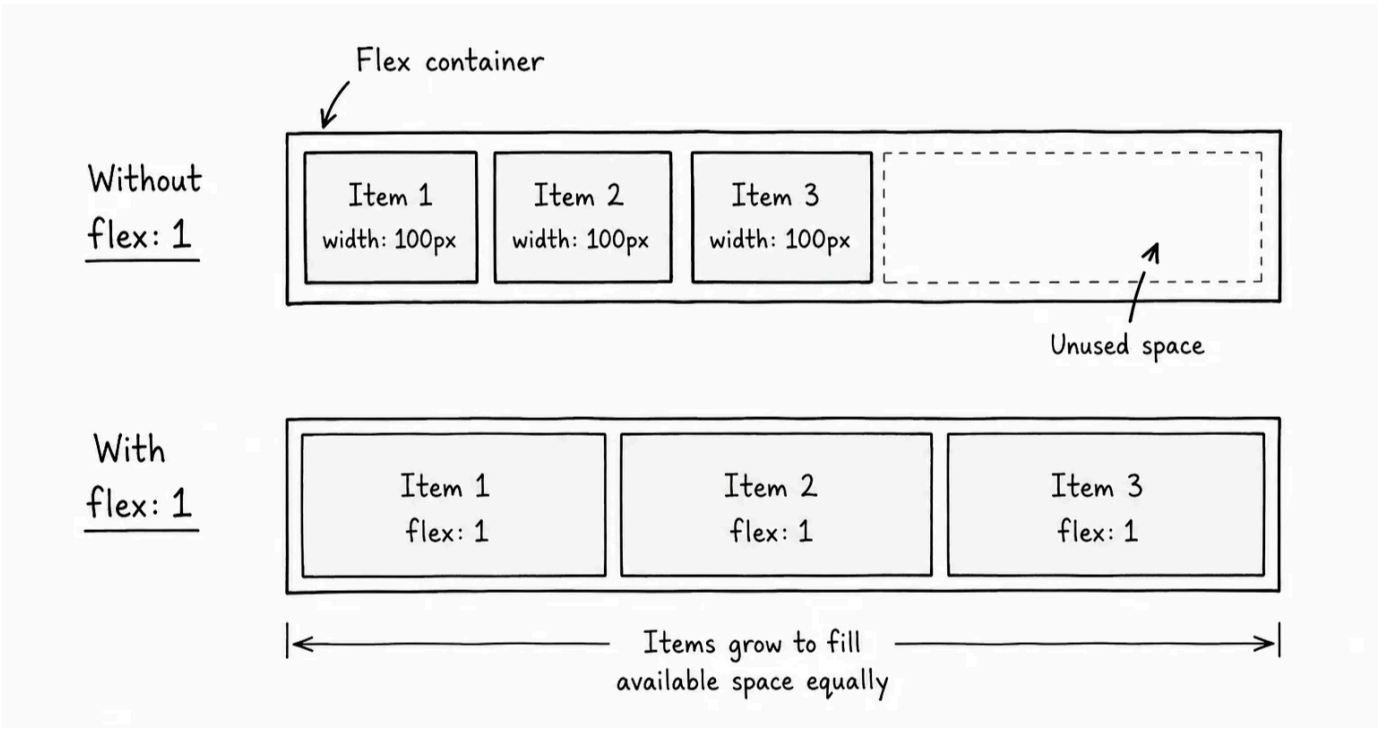
.lesson-card {
  flex: 1;
  min-width: 220px;
  margin-bottom: 0;
}
```

.lesson-list is the parent, so its direct children, the cards, become flex items.

flex: 1 tells each card it can grow and share the available space with the other cards. For now, read it as "let this item take an equal share of the row."

min-width: 220px protects each card from becoming too narrow. When there is not enough room for all cards to stay at least **220px** wide, **flex-wrap: wrap** lets the next card move to a new line.

margin-bottom: 0 removes the old vertical spacing from the stacked version. Once the cards are in a flex group, **gap** owns the spacing between them.



⚠ **Flexbox only arranges direct children** - If `display: flex` does not affect the elements you expected, inspect the HTML. You may have put Flexbox on the wrong parent. Only direct children of the flex container become flex items.

Align small rows inside a card

Flexbox is not only for big page sections. It is also useful for small rows inside components.

Each card has a footer with two direct children:

html

```
<div class="card-footer">
  <span>7 min</span>
  <a class="card-link" href="/flexbox">Open</a>
</div>
```

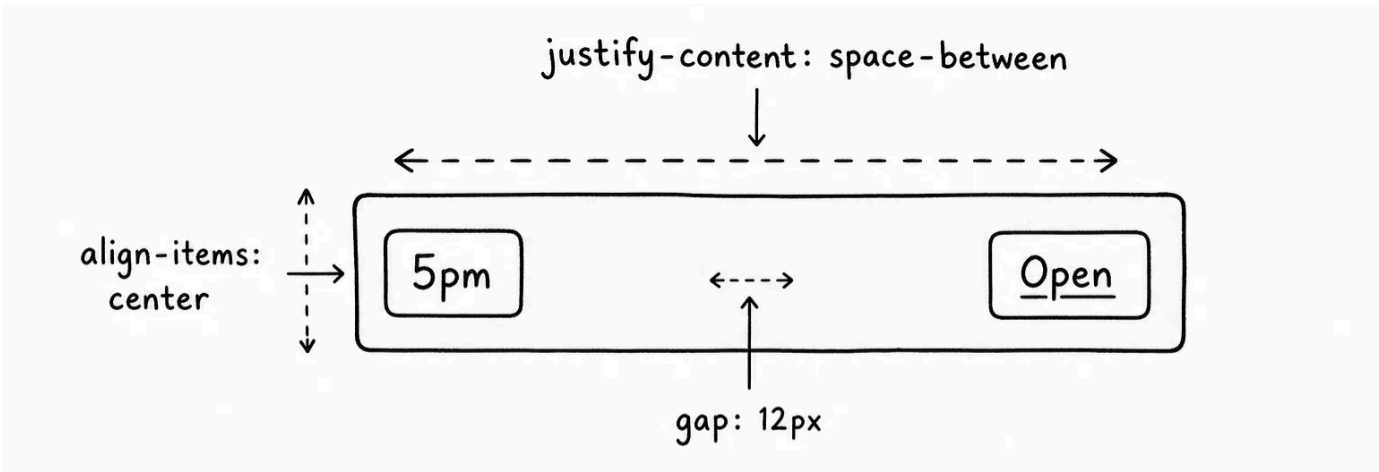
The time label and the link are direct children of `.card-footer`, so `.card-footer` is the parent that needs Flexbox:

CSS

```
.card-footer {
  display: flex;
  justify-content: space-between;
  align-items: center;
  gap: 12px;
  margin-top: 18px;
  color: #6b7280;
  font-size: 14px;
}
```

`justify-content: space-between` pushes the time label and link apart, `align-items: center` lines them up vertically, and `gap: 12px` keeps a little space between them if the row gets tight.

This is the same idea as the header, just on a smaller parent.



Row or column

Flexbox uses a row by default:

CSS

```
.toolbar {
  display: flex;
  flex-direction: row;
}
```

You can switch the main axis with **flex-direction** :

CSS

```
.toolbar {
  display: flex;
  flex-direction: column;
  gap: 8px;
}
```

flex-direction: column stacks the direct children vertically, but keeps Flexbox features like **gap** and alignment. Use a row when items should sit beside each other. Use a column when items should stack but still behave like a managed group.

Flexbox is best for one direction at a time. If you need strong control over rows and columns together, that is usually a grid layout problem, which is the next lesson.

Keep the HTML order sensible - CSS layout should not make the visual order tell a different story from the HTML order. Keyboard users, screen readers, and anyone reading the source still depend on a sensible document order.

Put the Flexbox rules together

Here are the Flexbox rules you added:

CSS

```
.page-header {
  display: flex;
}
```

```
justify-content: space-between;
align-items: flex-start;
gap: 16px;
}

.toolbar {
  display: flex;
  gap: 8px;
  flex-wrap: wrap;
  margin: 24px 0;
}

.lesson-list {
  display: flex;
  gap: 16px;
  flex-wrap: wrap;
  margin-top: 24px;
}

.lesson-card {
  flex: 1;
  min-width: 220px;
  margin-bottom: 0;
}

.card-footer {
  display: flex;
  justify-content: space-between;
  align-items: center;
  gap: 12px;
  margin-top: 18px;
  color: #6b7280;
  font-size: 14px;
}
```

The pattern is the same each time: choose the parent, decide whether its direct children should form a row or column, then use `gap` , `justify-content` , `align-items` , wrapping, or flexible item sizing to solve the layout problem.

 AI Tutor

I am learning Flexbox. Quiz me with small HTML examples and ask me which element should get display: flex. Then ask me whether the problem needs gap, justify-content, align-items, flex-wrap, or flex: 1. Focus on reasoning from the parent and direct children.

>

Try it

Test Flexbox by changing one layout decision at a time:

- Remove `display: flex` from `.page-header` . Notice that `justify-content` , `align-items` , and `gap` no longer create the header layout.
- Change `.page-header` from `justify-content: space-between;` to `justify-content: flex-start;` , then to `justify-content: center;` .
- Change `.page-header` from `align-items: flex-start;` to `align-items: center;` .
- Remove `flex-wrap: wrap;` from `.toolbar` , then make the browser narrower.

- Change `.lesson-card min-width` from `220px` to `300px` . Notice when the cards wrap.
- Temporarily remove `.lesson-card flex: 1;` . Notice that the cards stop sharing the row in the same way.
- Add a fourth `.lesson-card` and check whether the layout still works without writing a new class.
- Inspect `.lesson-list` in DevTools and confirm that the `.lesson-card` elements are the flex items.

The goal is to stop guessing at layout. When a group of items needs to line up, ask which parent owns the group, which children should be arranged, and which axis the layout should use.

< 0 / 11 >

🕒 Knew 0 Items

🌟 Learnt 0 Items

⏭ Skipped 0 Items

🔄 ResetProgress

Test Your Understanding

What does display: flex do?

[Click to Reveal the Answer](#)

🕒 Already Know that

🌟 Didn't Know that

⏭ Skip Question

✓

LESSON COMPLETE

Nicely done.

Up next: Personal Portfolio

Next Lesson →